



SEMINARIO MPI

DIEGO DÍAZ MATEOS
MARTA DE CASTRO LEIRA
MIGUEL BARRANQUERO DÍEZ
ÁLVARO BARCALA MARTÍN



ASPECTOS RELEVANTES

Limitación del array de números aleatorios: Se ha establecido una longitud máxima de 25 para el array de números aleatorios. Esto garantiza un rango controlado para los números que los procesos PG deben adivinar.

Manejo de la cantidad de procesos PG: Si el número de procesos PG es mayor que la cantidad de números a adivinar, algunos procesos PG no podrán adivinar. Esto se debe a que no hay suficientes números para que cada proceso PG tenga uno para adivinar.

Distribución desigual de números para adivinar: Si hay más números para adivinar que procesos PG, algunos procesos PG adivinarán más números que otros. Esto depende de la velocidad con la que cada proceso PG elija y adivine los números.

Requisitos mínimos de procesos: Se ha establecido que debe existir al menos un proceso de ejecución secuencial (PES), que será único en el sistema. Además, se requiere al menos un proceso generador (PG) y un proceso adivinador (PA) para el funcionamiento del sistema.



FUNCIONES UTILIZADAS:

- `double Wtime(void):`
- `MPI_Datatype getMPI_Estadisticas(Stats *stats)`
- `MPI_Datatype getMPI_FinalStats(FinalStats *finalStats)`
- `void cargarNumeros(int vectorNum[])`
- `void fuerza_espera(unsigned long peso)`
- `void process_pg(int rank, int size)`
- `void process_pes(int rank, int size, int pg_count, int pa_count)`
- `void process_pa(int rank, int size)`
- `void process_pi(int rank, int size)`



ESTRUCTURA DE COMUNICACIÓN:

Para comunicar el proceso 0 con los demás se han utilizado envíos y recepciones bloqueantes, pero con una comprobación previa no bloqueante MPI_PROBE. ESTO. Se han creado etiquetas para diferenciar los distintos tipos de mensajes

El PES envía un mensaje a todos los PGs con la etiqueta TAG_INFOPAS que contiene la información necesaria para cada PG, como el rango del espacio de búsqueda y la cantidad de números a adivinar. Los PGs reciben la tarea del PES con la etiqueta TAG_NOTIFICAR_NUMERO.

Los PGs envían los números que deben ser adivinados por los PAs a través de mensajes etiquetados con TAG_ADIVINAR. Después de adivinar un número, los PAs envían su respuesta al PG correspondiente utilizando la etiqueta TAG_PROCESAMIENTO_NUMERO.

Después de recibir todas las respuestas de los PAs, los PGs envían los resultados al PES con la etiqueta TAG_RESULTADO. Esto incluye el número adivinado y el tiempo empleado en la adivinanza.

Los PIs notifican al PES su finalización mediante mensajes etiquetados con TAG_FIN.

Se utilizan marcas de tiempo antes y después de cada proceso significativo para calcular el tiempo empleado en cada tarea. También Se realizan comprobaciones para garantizar la integridad de los datos y la coordinación adecuada entre los procesos.

CÓDIGO Y ANÁLISIS DE CADA PROCESO:

PES:

```
// Comprobar si hay suficientes procesos para distribuir según la elección
if (pg_count + pa_count + 1 > size) {
    fprintf(stderr, "No hay suficientes procesos para distribuir según tu elección.\n");
    MPI_Abort(MPI_COMM_WORLD, 1);
    MPI_Finalize();
    exit(1);
}
```

```
// Notificar a los procesos PG sobre su tipo
for (int i = 1; i <= pg_count; i++) {
    MPI_Send(&i, 1, MPI_INT, i, TAG_ID, MPI_COMM_WORLD);
    printf("\tProceso PG %d\n", i);
}

// Notificar a los procesos PA sobre su tipo
for (int i = pg_count + 1; i <= pg_count + pa_count; i++) {
    MPI_Send(&i, 1, MPI_INT, i, TAG_ID, MPI_COMM_WORLD);
    printf("\tProceso PA %d\n", i);
}

// Notificar a los procesos PI sobre su tipo
for (int i = pg_count + pa_count + 1; i < size; i++) {
    MPI_Send(&i, 1, MPI_INT, i, TAG_ID, MPI_COMM_WORLD);
    printf("\tProceso PI %d\n", i);
}

printf("=====n");

for(i=0; i<pg_count;i++){
    estadosPGs[i][0]= LIBRE;
    estadosPGs[i][1]= 1 + i;
}

infoPaToPg[0] = pa_count;
for(int i = 1 ; i <= pa_count; i++){
    infoPaToPg[i] = pg_count + i ;
}

//Envia a los PGs la informacion de los PAs
for (int i = 1; i <= pg_count; i++) {
    MPI_Send(&infoPaToPg, ARRAYSIZ, MPI_INT, i, TAG_INFOPAS, MPI_COMM_WORLD);
}

cargarNumeros(arrayNumeros);

if(pg_count > TOTALNUM)
    pg_real = TOTALNUM;
else
    pg_real = pg_count;
```

```
// Envío inicial de números a los PGs
for (int i = 0; i < pg_count; i++) {
    if (nextNumber < TOTALNUM) {
        MPI_Send(&arrayNumeros[nextNumber], 1, MPI_INT, estadosPGs[i][1], TAG_NOTIFICAR_NUMERO, MPI_COMM_WORLD);
        nextNumber++;
    }
}

while (numRemaining > 0 || statsRecv < size-1) {
    MPI_Probe(MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
    int source = status.MPI_SOURCE;
    int tag = status.MPI_TAG;
```

```
// Recepción de estadísticas parciales
if (tag == TAG_ESTADISTICAS) {
    MPI_Recv(&stats_parcial, 1, structEstadisticas, source, TAG_ESTADISTICAS, MPI_COMM_WORLD, &status);
    recibidosParciales[recibidos] = stats_parcial;
    recibidos++;
    numRemaining--;
}

// Si quedan números en la lista, envía el siguiente
if (nextNumber < TOTALNUM) {
    MPI_Send(&arrayNumeros[nextNumber], 1, MPI_INT, source, TAG_NOTIFICAR_NUMERO, MPI_COMM_WORLD);
    nextNumber++;
} else {
    // Si no quedan números y ya se encontraron todos, envía un mensaje de finalización al PG, PI, PA
    contadorFinalizados++;
    if(contadorFinalizados == pg_real){
        for (int i = 1; i < pg_count; i++) {
            MPI_Send(NULL, 0, MPI_INT, i, TAG_FINALIZACION, MPI_COMM_WORLD);
        }
        for (int i = pg_count + pa_count + 1; i < size; i++) {
            MPI_Send(NULL, 0, MPI_INT, i, TAG_FINALIZACION, MPI_COMM_WORLD);
        }
        for (int i = pg_count + 1; i <= pg_count + pa_count; i++) {
            MPI_Send(NULL, 0, MPI_INT, i, TAG_FINALIZACION, MPI_COMM_WORLD);
        }
        for (int i = pg_real; i <= pg_count; i++) {
            MPI_Send(NULL, 0, MPI_INT, i, TAG_FINALIZACION, MPI_COMM_WORLD);
        }
    }
}

} else if (tag == TAG_FINAL){
    statsRecv++;
    MPI_Recv(&stats_final, 1, structEstadisticasFinal, source, TAG_FINAL, MPI_COMM_WORLD, &status);
    recibidosTotales[source] = stats_final;
}
```

CÓDIGO Y ANÁLISIS DE CADA PROCESO:

PG:

```
// Recibir información del tipo y el ID del proceso desde el PES
MPI_Recv(&id, 1, MPI_INT, PES, TAG_ID, MPI_COMM_WORLD, &status);
tiempo_tot_ini = Wtime();
tiempo_totF_ini = Wtime();
tiempo_inicio_procesamiento = Wtime();
contador_mpi_r++;

// Verificar que el mensaje proviene del PES
if (status.MPI_SOURCE != PES) {
    fprintf(stderr, "Error: Mensaje recibido no proviene del PES.\n");
    MPI_Abort(MPI_COMM_WORLD, 1);
}
```

```
// Recibir información de los PAs enviada desde PES
MPI_Recv(&auxInfoPAs, ARRAYSZ, MPI_INT, PES, TAG_INFOPAS, MPI_COMM_WORLD, &status);
contador_mpi_r++;
totalPAs = auxInfoPAs[0];
int idPAs[totalPAs];

for(i = 1; i <= totalPAs; i++){
    idPAs[i-1] = auxInfoPAs[i];
}

while(i){
    contador=0;
    MPI_Probe(MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
    contador_mpi_p++;
    tag = status.MPI_TAG;
    //Recibir número para adivinar
    if(tag == TAG_NOTIFICAR_NUMERO){
        MPI_Recv(&numAdivinar, 1, MPI_INT, PES, TAG_NOTIFICAR_NUMERO, MPI_COMM_WORLD, &status);
        contador_mpi_r++;
        tiempo_inicio_procesamiento_pg=0.0;
        tiempo_inicio_procesamiento_pg = Wtime();
        double tiempo_calculo_numero_pg = 0.0;
        tiempo_calculo_numero_pg = Wtime();
        stats_parcial.numero=numAdivinar;
        int idPA;
        while(buscando_conexion == FALSE){
            if(k == totalPAs){
                if(k == 0;){
                    MPI_Send(&solicitarPA, 1, MPI_INT, idPAs[k], TAG_SOLICITUD_CONEXION, MPI_COMM_WORLD);
                    contador_mpi_s++;
                    MPI_Recv(&buscando_conexion, 1, MPI_INT, idPAs[k], TAG_SOLICITUD_CONEXION, MPI_COMM_WORLD, &status);
                    contador_mpi_r++;
                    if (buscando_conexion == FALSE) {
                        contador_pa_ocupado++;
                    }
                    if(buscando_conexion == TRUE){
                        conexion_realizada_PA = TRUE;
                        idPA = k;
                        break;
                    }
                    k = k+1;
                }
            }
            k++;
        }
    }
}
```

```
while(conexion_realizada_PA == TRUE){

    MPI_Recv(&numMedio, 1, MPI_INT, idPAs[idPA], TAG_PROCESAMIENTO_NUMERO, MPI_COMM_WORLD, &status);
    contador_mpi_r++;
    contador++;

    if(numAdivinar>numMedio){
        caracter = MAYOR; //">"
        fuerza_espera(PESO_MIM);
        MPI_Send(&caracter, 1, MPI_INT, idPAs[idPA], TAG_PROCESAMIENTO_NUMERO, MPI_COMM_WORLD);
        contador_mpi_s++;
    }else if (numAdivinar<numMedio){
        caracter = MENOR; //"<"
        fuerza_espera(PESO_MIM);
        MPI_Send(&caracter, 1, MPI_INT, idPAs[idPA], TAG_PROCESAMIENTO_NUMERO, MPI_COMM_WORLD);
        contador_mpi_r++;
    }else{
        caracter = IGUAL; //"="
        fuerza_espera(PESO_MIM);
        MPI_Send(&caracter, 1, MPI_INT, idPAs[idPA], TAG_PROCESAMIENTO_NUMERO, MPI_COMM_WORLD);
        contador_mpi_r++;
        // Medir tiempo de fin de procesamiento
        double tiempo_fin_procesamiento_pg = 0.0;
        tiempo_fin_procesamiento_pg = Wtime();
        stats_parcial.tiempo_calculo = tiempo_fin_procesamiento_pg - tiempo_inicio_procesamiento_pg;

        conexion_realizada_PA = FALSE;
        buscando_conexion = FALSE;

        double tiempo_tot_fin = 0.0;
        tiempo_tot_fin = Wtime();
        stats_parcial.tiempo_total = tiempo_tot_fin - tiempo_tot_ini;

        stats_parcial.respuestas=contador;
        stats_parcial.consultas_ocupado= contador_pa_ocupado;
        stats_parcial.numero_pg = rank;

        MPI_Send(&stats_parcial, 1, structEstadisticas, PES, TAG_ESTADISTICAS, MPI_COMM_WORLD);
        contador_mpi_s++;
        tiempo_inicio_procesamiento_pg= Wtime();
        tiempo_tot_ini=0.0;
        tiempo_tot_ini = Wtime();
    }
}
```

CÓDIGO Y ANÁLISIS DE CADA PROCESO:

PA:

```
// Recibir información del tipo y el ID del proceso desde el PES
MPI_Recv(&id, 1, MPI_INT, PES, TAG_ID, MPI_COMM_WORLD, &status);
double tiempo_inicio_procesamiento = Wtime();
contador_mpi_r++;

if (status.MPI_SOURCE != PES) {
    fprintf(stderr, "Error: Mensaje recibido no proviene del PES.\n");
    MPI_Abort(MPI_COMM_WORLD, 1);
}
```

```
//Recibe la solicitud de conexion de un PG, En caso de estar libre la acepta
while(1){
    MPI_Probe(MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
    contador_mpi_p++;
    tag = status.MPI_TAG;
    origenSolicitud = status.MPI_SOURCE;
    if(tag == TAG_SOLICITUD_CONEXION){
        MPI_Recv(&conexion, 1, MPI_INT, origenSolicitud, TAG_SOLICITUD_CONEXION, MPI_COMM_WORLD, &status);
        contador_mpi_r++;

        if(estatusPA == LIBRE){
            MPI_Send(&estatusPA, 1, MPI_INT, origenSolicitud, TAG_SOLICITUD_CONEXION, MPI_COMM_WORLD);
            contador_mpi_s++;
            estatusPA = OCUPADO;
            origenPG = origenSolicitud;
            numMax=999999;
            numMin=0;
            numMedio = (numMin +(numMax-numMin)/2);
            //Empieza el procesamiento del dato
            MPI_Send(&numMedio, 1, MPI_INT, origenPG, TAG_PROCESAMIENTO_NUMERO, MPI_COMM_WORLD);
            tiempo_calculos_inicio=Wtime();

            contador_mpi_s++;
        }else{
            MPI_Send(&estatusPA, 1, MPI_INT, origenSolicitud, TAG_SOLICITUD_CONEXION, MPI_COMM_WORLD);
            contador_mpi_s++;
        }
    }
}
```

```
}
}else if(tag == TAG_PROCESAMIENTO_NUMERO){
    MPI_Recv(&caracter, 1, MPI_INT, origenPG, TAG_PROCESAMIENTO_NUMERO, MPI_COMM_WORLD, &status);
    tiempo_qa_fin =Wtime();

    contador_mpi_r++;
    if(caracter == MAYOR){
        numMin = numMedio;
        numMedio = (numMin +(numMax-numMin)/2);
        fuerza_espera(PESO_MEDIO);
        tiempo_qa_ini= Wtime();
        MPI_Send(&numMedio, 1, MPI_INT, origenPG, TAG_PROCESAMIENTO_NUMERO, MPI_COMM_WORLD);
        contador_mpi_s++;
    }else if(caracter == MENOR){
        numMax= numMedio;
        numMedio = (numMin +(numMax-numMin)/2);
        fuerza_espera(PESO_MEDIO);
        tiempo_qa_ini= Wtime();
        MPI_Send(&numMedio, 1, MPI_INT, origenPG, TAG_PROCESAMIENTO_NUMERO, MPI_COMM_WORLD);
        contador_mpi_s++;
    }else if(caracter == IGUAL){
        estatusPA = LIBRE;
        fuerza_espera(PESO_MEDIO);
        tiempo_calculos_fin=Wtime();
        tiempo_calculos+= (tiempo_calculos_fin-tiempo_calculos_inicio);
    }
    if(tiempo_qa < (tiempo_qa_fin-tiempo_qa_ini))
        tiempo_qa = tiempo_qa_fin-tiempo_qa_ini;
}

}else if(tag == TAG_FINALIZACION){
    //ACABA PA y envia estadísticas a PES
    MPI_Recv(NULL, 0, MPI_INT, PES, TAG_FINALIZACION , MPI_COMM_WORLD, &status);
    contador_mpi_r++;

    contador_mpi_s++;
    stats_final.mpi_probe=contador_mpi_p;
    stats_final.mpi_recive=contador_mpi_r;
    stats_final.mpi_send=contador_mpi_s;
    double tiempo_fin_procesamiento = Wtime();
    stats_final.tiempo_total = tiempo_fin_procesamiento - tiempo_inicio_procesamiento;
    stats_final.tiempo_calculo=tiempo_calculos;
    stats_final.tiempo_max_qa_pg = tiempo_qa;

    MPI_Send(&stats_final, 1, structEstadisticasFinal, PES, TAG_FINAL, MPI_COMM_WORLD);

    return; //FINALIZA PA
}
```



CÓDIGO Y ANÁLISIS DE CADA PROCESO:

PI:

```
// Recibir información del tipo y el ID del proceso desde el PES
MPI_Recv(&id, 1, MPI_INT, PES, TAG_ID, MPI_COMM_WORLD, &status);
double tiempo_inicio_procesamiento = Wtime();
contador_mpi_r++;

if (status.MPI_SOURCE != PES) {
    fprintf(stderr, "Error: Mensaje recibido no proviene del PES.\n");
    MPI_Abort(MPI_COMM_WORLD, 1);
}

MPI_Recv(NULL, 0, MPI_INT, PES, TAG_FINALIZACION, MPI_COMM_WORLD, &status);
contador_mpi_r++;
contador_mpi_s++;
stats_final.mpi_probe=contador_mpi_p;
stats_final.mpi_recive=contador_mpi_r;
stats_final.mpi_send=contador_mpi_s;
double tiempo_fin_procesamiento = Wtime();
stats_final.tiempo_total = tiempo_fin_procesamiento - tiempo_inicio_procesamiento;

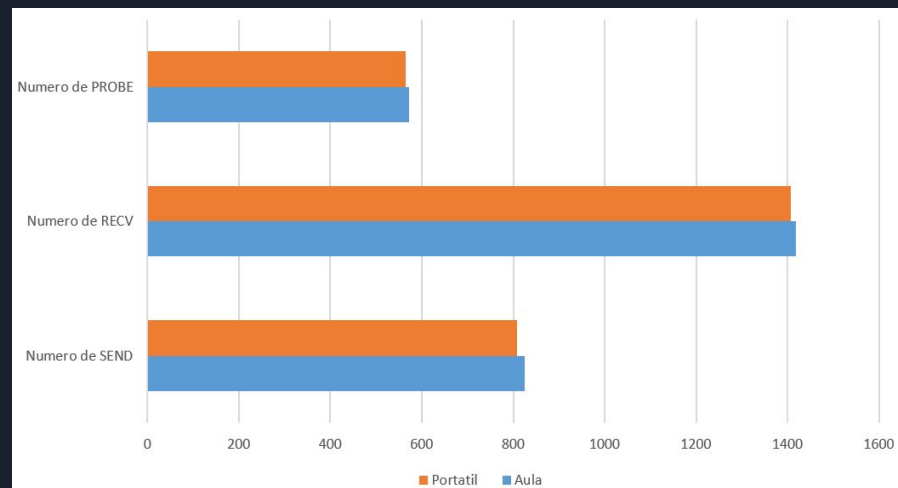
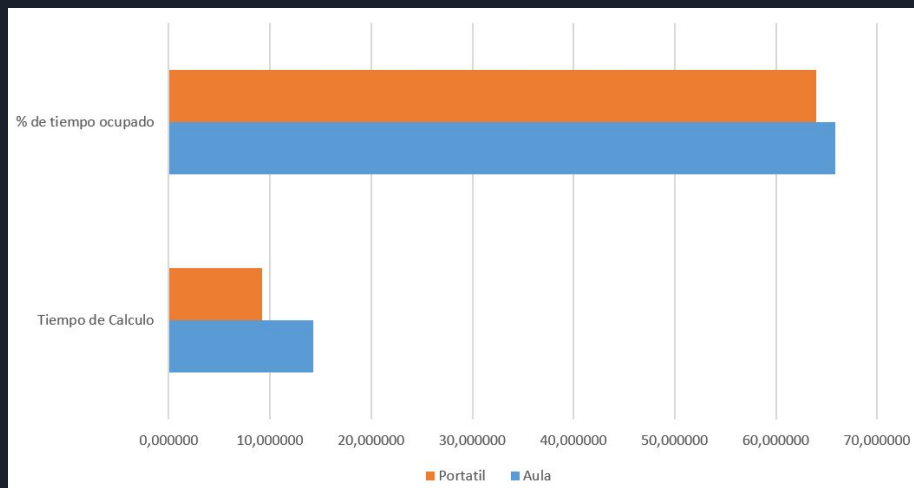
MPI_Send(&stats_final, 1, structEstadisticasFinal, PES, TAG_FINAL, MPI_COMM_WORLD);
return;//FINALIZA PI
```



ANÁLISIS DE RESULTADOS:

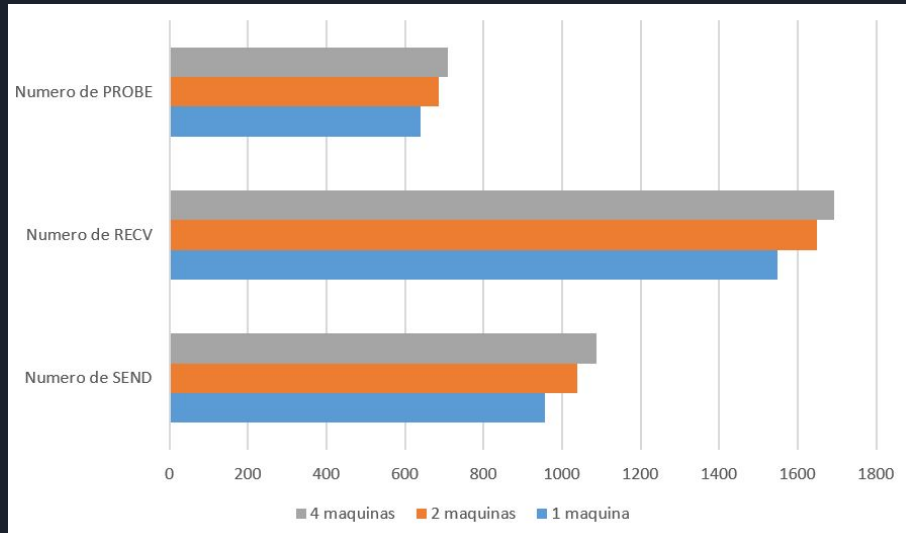
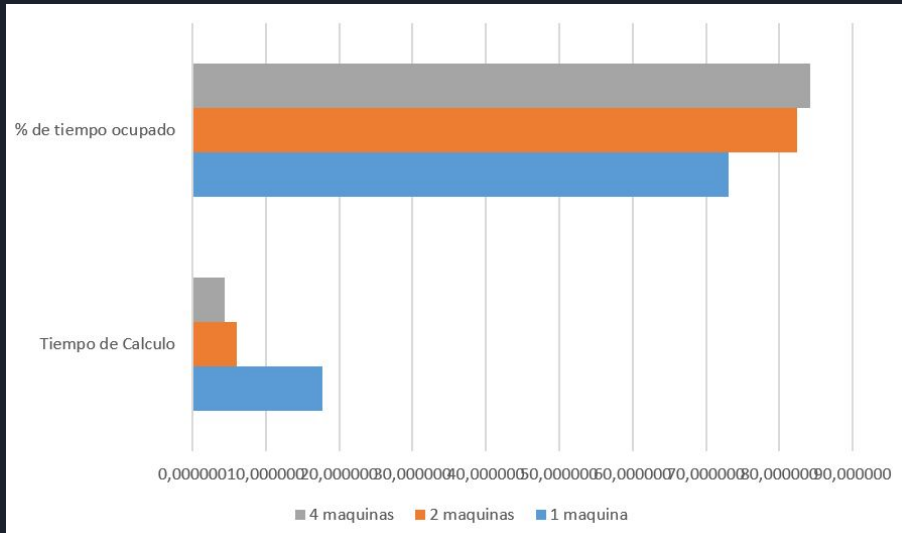
DOS MÁQUINAS CON DIFERENTES PROCESADORES

	PROCESOS	GESTORES	ADIVINADORES	T. CÁLCULO	% CÁLCULO	SEND	RECV	PROBE
aula	10	3	3	14,299042	65,84	825	1419	572
portatil	10	3	3	9,251712	63,99	809	1407	565



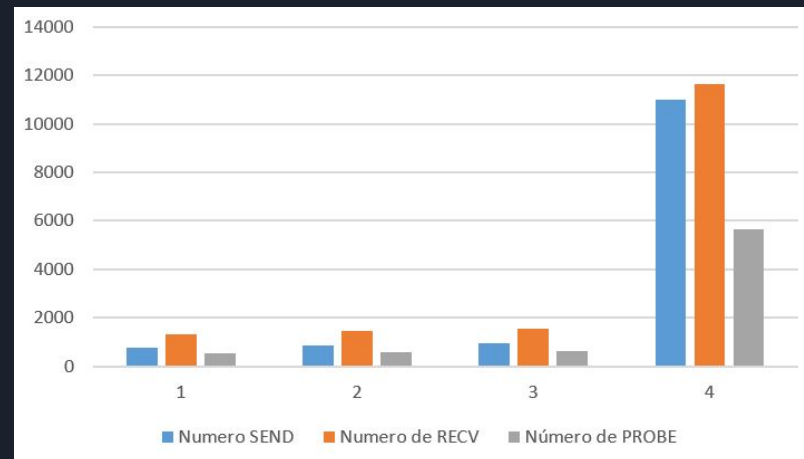
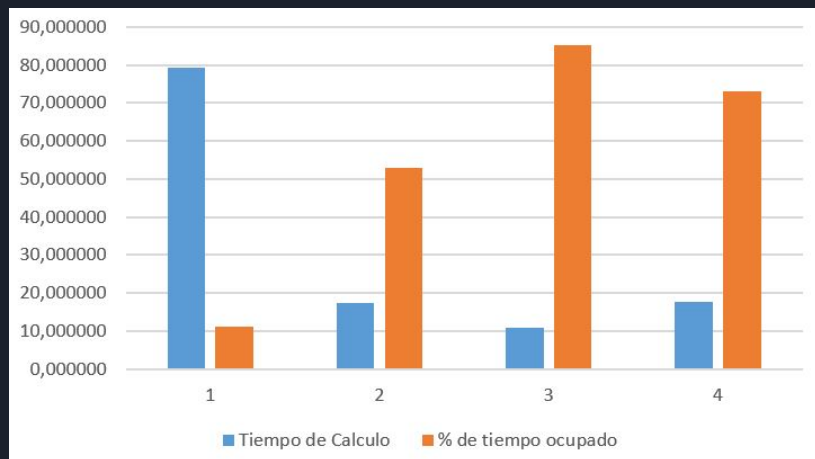
1 MÁQUINA VS. 2 MÁQUINAS VS. 4 MÁQUINAS

	PROCESOS	GESTORES	ADIVINADORES	T. CÁLCULO	% CÁLCULO	SEND	RECV	PROBE
1 máquina	19	9	9	17,755347	73,13	957	1548	641
2 máquinas	19	9	9	6,051122	82,42	1039	1650	687
4 máquinas	19	9	9	4,369394	84,16	1087	1694	710



DIFERENTES DISTRIBUCIONES DE TIPOS DE PROCESO

PROCESOS	GESTORES	ADIVINADORES	T. CÁLCULO	% CÁLCULO	SEND	RECV	PROBE
19	1	1	79,309858	11,07	753	1340	524
19	5	10	17,223618	52,98	840	1441	583
19	9	9	10,822707	85,08	957	1548	641
19	10	5	17,755347	73,13	11012	11622	5670





CONCLUSIONES:

Podemos concluir que el reparto de la carga en varias máquinas mejore notablemente el rendimiento, disminuyendo el tiempo de cálculo e incrementando el % de tiempo ocupado en el cálculo.

También hemos llegado a la conclusión de que con un número alto de procesos inactivos PI, el rendimiento se desploma. Se alcanza el punto más eficiente cuando se igualan el número de PAs y PGs y no hay ningún proceso PI, obteniendo aquí los mejores resultados.

Respecto a la cantidad de SEND RECV y PROBE no varía excesivamente al utilizar varias máquinas siendo esta ligeramente superior que cuando se utiliza solo una. Notamos como esta cantidad se dispara cuando el número de PG es mayor que el de PA